Lab 3 - DLX Fetch

Nate Herbst
A02307138
Nathan Walker
A02364124

Introduction

This project required making the fetch stage out of the 5 stages for the DLX processor. This stage has several components including: registers, Muxes, adders, and ROMs. This project entails incrementing the program counter so that the decode stage would receive the instruction at the current PC value, and the PC + 1 value. With the option to have a jump address selected from the execute stage of the processor.

Procedure

Modular Systems:

- We started off by making each block that would match the blocks we had made in our block diagram. (See Figure 1)
- We then tied them together
- We then made each test bench to verify each smaller piece of the bigger puzzle. (See Figures 2-4)
- Created _pkg files for each component to ease the use of them in later labs if needed

Challenges and Solutions:

- The primary difficulty lay in simulating the top-level `fetch.vhd` file within Questa. Lacking the complete soft-core processor, specifically the `Decode` and `Execute` stages, made it challenging to accurately model the source of the next program counter or jump instruction. This required significant effort to determine the correct clock cycle delays, but we successfully developed the necessary testbench to achieve a working simulation.

Results

The final implementation showed us that the fetch stage can correctly increment the PC and grab correct instructions. It can also be selected to jump to a certain instruction address and properly increment the counter from the jump. The Fetch stage was made in a modular fashion so in the future it can be easily changed and portions can easily be ported to other sections.

Figures

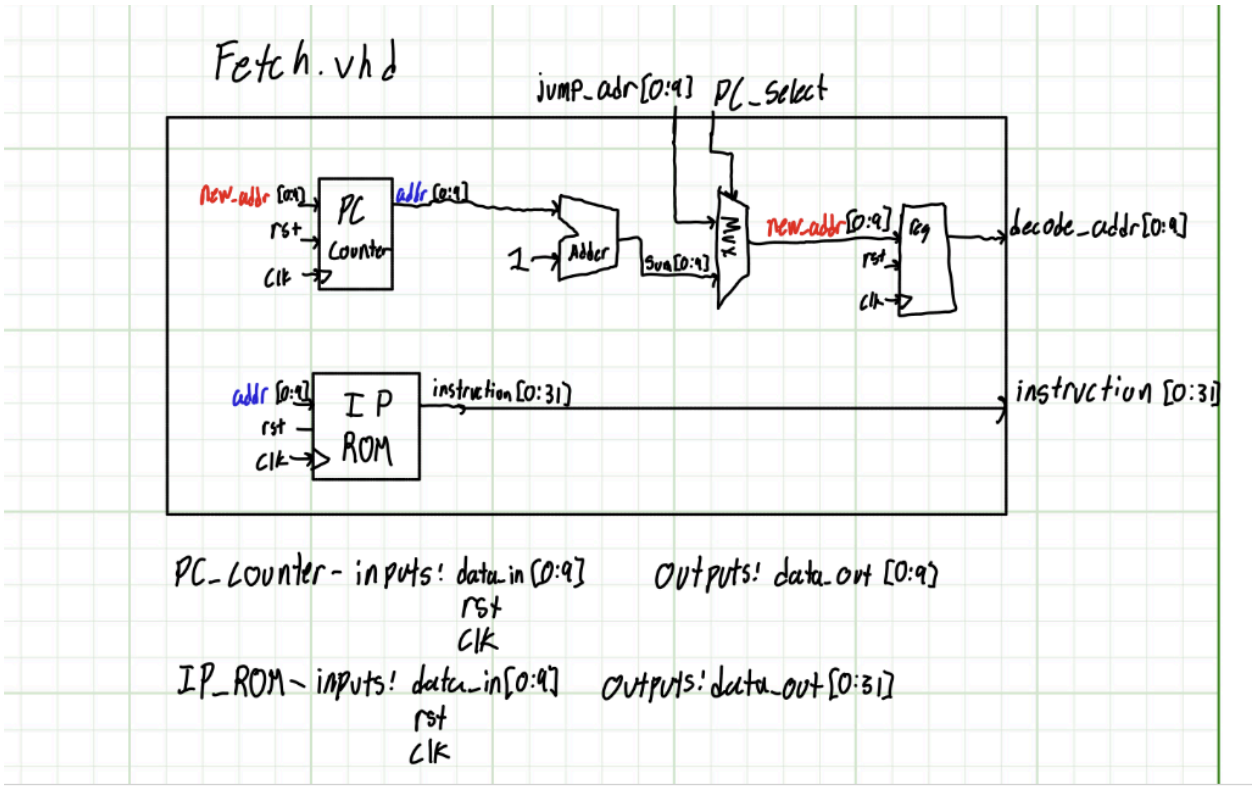

Figure 1: First Initial Block diagram for all the blocks we needed to implement and link together

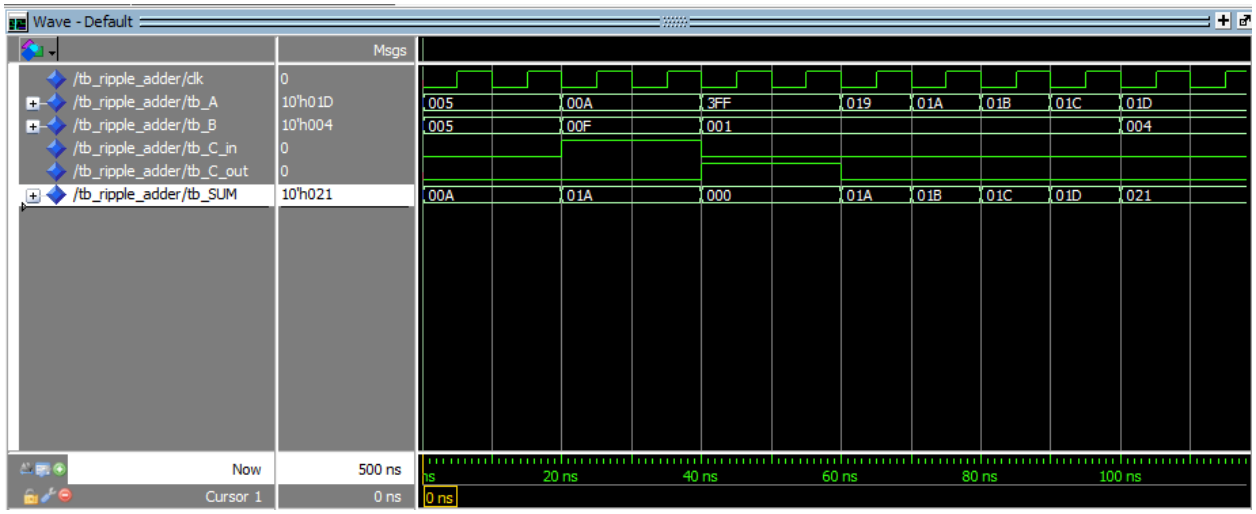

Figure 2: Ripple adder simulation

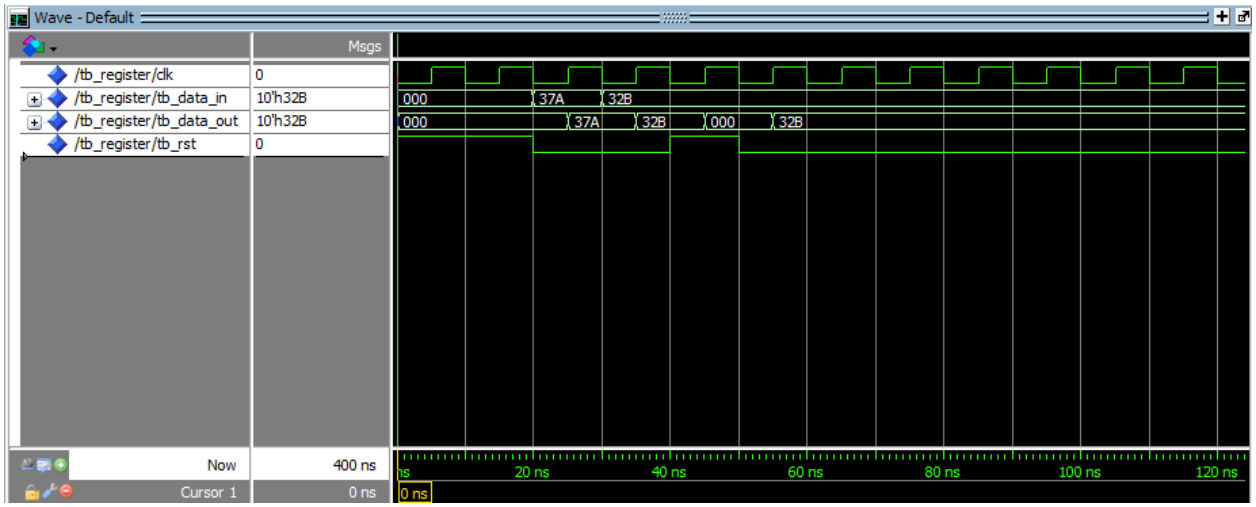

Figure 3: Register simulation

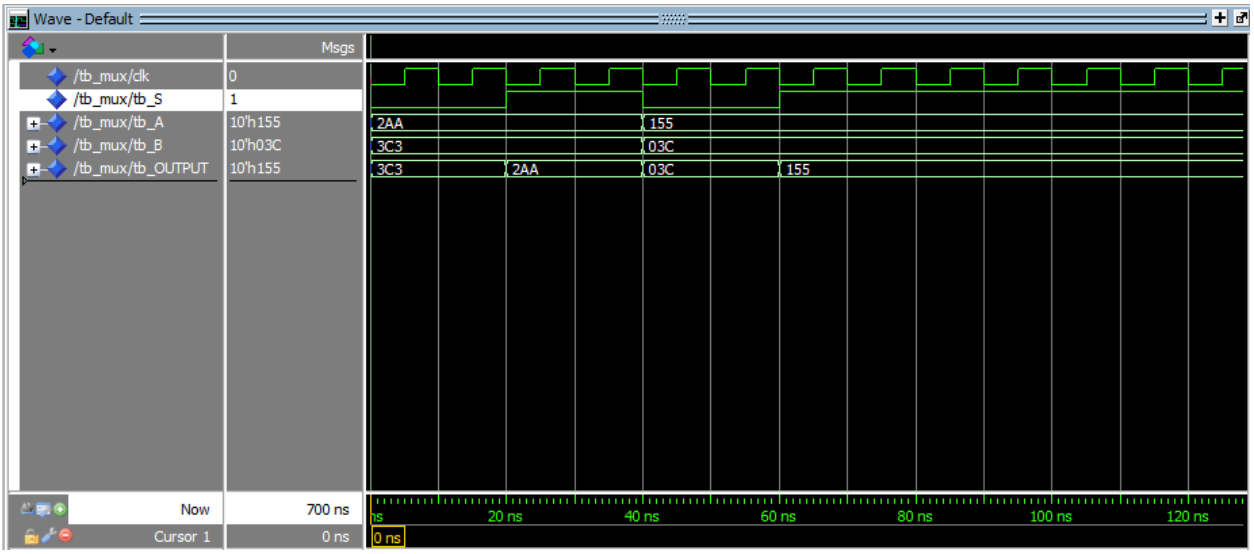

Figure 4: MUX simulation

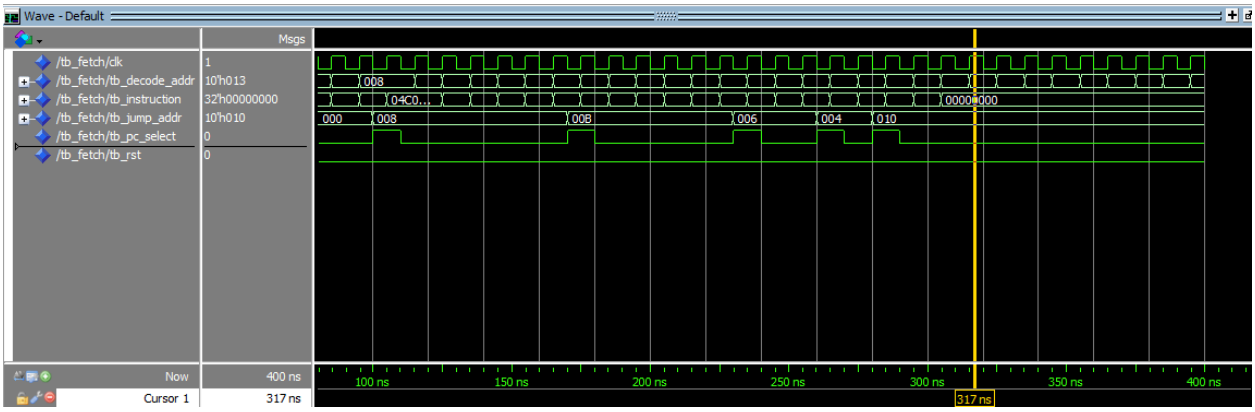

Figure 5:Sim snapshot of the fetch stage working

Conclusion

This lab provided valuable insights into the workflow on implementing different stages of a processor. This lab will help us in the future labs as we have a better understanding of how to pipeline the data and when and where to register the data. It has been a fun experience and we are learning a lot of hardware design flow through this.

Appendix

fetch.vhd (top level file)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;
--use work.register_pkg.all;
--use work.MUX_pkg.all;
--use work.ripple_adder_pkg.all;

entity fetch is
   generic (
       N   :   integer := 10;
       M   :   integer :=  32
   );
   port    (
       jump_addr   :   in      std_logic_vector(N-1 downto 0);
       pc_select   :   in      std_logic;
       rst         :   in      std_logic;
       clk         :   in      std_logic;
       decode_addr :   out std_logic_vector(N-1 downto 0);
       instruction :   out std_logic_vector(M-1 downto 0)
   );
end entity;

architecture component_list of fetch is

   signal  new_addr    :   std_logic_vector(N-1 downto 0) := (others =>
'0');
   signal  addr        :   std_logic_vector(N-1 downto 0) := (others =>
'0');
   signal  sum     :   std_logic_vector(N-1 downto 0) := (others => '0');
   signal  C_DUMMY :   std_logic;

   constant LSB_ONE    :   std_logic_vector(N-1 downto 0) :=
     (N-2 downto 0 => '0') & '1';
```

```
   constant ZERO       :   std_logic := '0';

begin

   PC_counter  :   entity work.reggi
       generic map(
           N => 10
       )
       port map(
           data_in     => new_addr,
           rst         =>  rst,
           clk     =>  clk,
           data_out    =>  addr
       );

   ADDER       :   entity work.ripple_adder
       generic map(
           N => 10
       )
       port map(
           A       => addr,
           B       => LSB_ONE,
           C_in    => ZERO,
           SUM => sum,
           C_out => C_DUMMY
       );

   MUXXY       :   entity work.MUX
       generic map(
           N => 10
       )
       port map(
           A       =>  jump_addr,
           B       => sum,
           S       => pc_select,
           OUTPUT=>    new_addr
       );

   MUX_REGISTER    :   entity work.reggi
       generic map(
```

```
            N => 10
        )
        port map(
            data_in => new_addr,
            rst     =>  rst,
            clk     =>  clk,
            data_out    =>  decode_addr
        );

    --insert IP ROM device with .mif file
    IMEM        :   entity work.factorial_ROM
        port map(
            address =>  addr,
            clock   =>  clk,
            q       =>  instruction
        );

end architecture component_list;
```

tb_fetch.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity tb_fetch is
end tb_fetch;

architecture behavioral of tb_fetch is

    component fetch
        generic (
            N   :   integer := 10;
            M   :   integer :=  32
        );
        port (
            jump_addr   :   in      std_logic_vector(N-1 downto 0);
            pc_select   :   in      std_logic;
            rst         :   in      std_logic;
```

```
            clk         :   in      std_logic;
            decode_addr :   out std_logic_vector(N-1 downto 0);
            instruction :   out std_logic_vector(M-1 downto 0)
        );
    end component fetch;

    -- Clock
    signal clk : std_logic;
    constant CLK_PERIOD : time := 10 ns;

    -- Testbench signals
    signal tb_jump_addr : std_logic_vector(9 downto 0) := (others => '0');
    signal tb_pc_select : std_logic := '0';
    signal tb_rst : std_logic := '1';
    signal tb_decode_addr : std_logic_vector(9 downto 0);
    signal tb_instruction : std_logic_vector(31 downto 0);

begin

    -- Instantiate the fetch stage
    uut : fetch
        generic map (
            N => 10,
            M => 32
        )
        port map(
            jump_addr   => tb_jump_addr,
            pc_select   => tb_pc_select,
            rst         => tb_rst,
            clk         => clk,
            decode_addr => tb_decode_addr,
            instruction => tb_instruction
        );

    -- Clock process
    clk_process : process
    begin
        clk <= '0';
        wait for CLK_PERIOD / 2;
        clk <= '1';
```

```
        wait for CLK_PERIOD / 2;
    end process;

    -- Stimulus process for factorial program
    stm_process : process
    begin
        -- 1. Reset the system to start at address 0
        tb_rst <= '1';
        wait for CLK_PERIOD * 2;
        tb_rst <= '0';
        wait for CLK_PERIOD;

        -- 2. Fetch instructions 0x000 through 0x004. At 0x004 (BEQZ),
assume branch not taken.
        tb_pc_select <= '0';
        wait for CLK_PERIOD * 5;

        -- 3. At PC=0x005, instruction is JAL factorial (0x008). Jump
there.
          wait for CLK_PERIOD*2;
        tb_pc_select <= '1';
        tb_jump_addr <= "0000001000"; -- Jump to address 0x008
        wait for CLK_PERIOD;
        tb_pc_select <= '0';
        wait for CLK_PERIOD;

        -- 4. Fetch 0x008, 0x009, 0x00A, 0x00B.
        wait for CLK_PERIOD * 4;

        -- 5. At PC=0x00C, instruction is SUBI. Fetch it.
        wait for CLK_PERIOD;

        -- 6. At PC=0x00D, instruction is BNEZ multiply_loop (0x00B).
Simulate branch taken.
        tb_pc_select <= '1';
        tb_jump_addr <= "0000001011"; -- Jump to address 0x00B
        wait for CLK_PERIOD;
        tb_pc_select <= '0';
        wait for CLK_PERIOD;
```

```
        -- 7. We are back at 0x00B. Fetch 0x00B, 0x00C.
        wait for CLK_PERIOD * 2;

        -- 8. At PC=0x00D, BNEZ again. This time, simulate branch NOT
taken.
        wait for CLK_PERIOD;

        -- 9. At PC=0x00E, instruction is SW. Fetch it.
        wait for CLK_PERIOD;

        -- 10. At PC=0x00F, instruction is JR R31. Return address was
0x005+1=0x006. Jump there.
        tb_pc_select <= '1';
        tb_jump_addr <= "0000000110"; -- Jump to address 0x006
        wait for CLK_PERIOD;
        tb_pc_select <= '0';
        wait for CLK_PERIOD;

        -- 11. At PC=0x006, instruction is SUBI. Fetch it.
        wait for CLK_PERIOD;

        -- 12. At PC=0x007, instruction is J loop (0x004). Jump there.
        tb_pc_select <= '1';
        tb_jump_addr <= "0000000100"; -- Jump to address 0x004
        wait for CLK_PERIOD;
        tb_pc_select <= '0';
        wait for CLK_PERIOD;

        -- 13. At PC=0x004, BEQZ again. This time, simulate branch TAKEN to
'done' (0x010).
        tb_pc_select <= '1';
        tb_jump_addr <= "0000010000"; -- Jump to address 0x010
        wait for CLK_PERIOD;
        tb_pc_select <= '0';
        wait for CLK_PERIOD;

        -- 14. At PC=0x010, instruction is J done (itself). Fetch it.
        wait for CLK_PERIOD * 2;

        wait;
```

```
    end process;

end architecture behavioral;
```

ripple_adder.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;
use work.full_adder_pkg.all;

entity ripple_adder is
    generic (
        N : integer := 10
    );
    port (
        A       : in std_logic_vector(N-1 downto 0);
        B       : in std_logic_vector(N-1 downto 0);
        C_in    : in std_logic;
        SUM : out std_logic_vector(N-1 downto 0);
        C_out   : out std_logic
    );
end entity ripple_adder;

architecture rtl of ripple_adder is

    signal C    : std_logic_vector(N downto 0);

begin

    C(0) <= C_in;

    gen_adders : for i in 0 to N-1 generate
        FA  :   entity work.full_adder
            port map(
                A       =>  A(i),
                B       =>  B(i),
```

```
            C_in    => C(i),
            SUM     => SUM(i),
            CARRY => C(i+1)
        );
  end generate gen_adders;

  C_out <= C(N);

end architecture rtl;
```

register.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity reggi is
  generic (
      N   :   integer := 10
  );
  port    (
      data_in :   in  std_logic_vector(N-1 downto 0);
      rst     :   in      std_logic;
      clk     :   in      std_logic;
      data_out    :   out std_logic_vector(N-1 downto 0)
  );
end entity reggi;

architecture behavioral of reggi is

  signal output_data  :   std_logic_vector(N-1 downto 0)
  := (others => '0');

begin

  data_out <= output_data;
  process(clk) begin
      if rising_edge(clk) then
          if rst = '1' then
```

```
                output_data <= (others => '0');
            else
                output_data <= data_in;
            end if;
        end if;
    end process;

end architecture behavioral;
```

MUX.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity MUX is
    generic (
        N : integer := 10
    );
    port (
        A       :   in      std_logic_vector(N-1 downto 0);
        B       :   in  std_logic_vector(N-1 downto 0);
        S       :   in  std_logic;
        OUTPUT: out     std_logic_vector(N-1 downto 0)
    );
end entity MUX;

architecture rtl of MUX is

begin

    OUTPUT <= A when S = '1' else B;

end architecture rtl;
```